Programmation de spécialité (Python)

Projet « Moteur de recherche » — Consignes générales

Master 1 Informatique (par alternance)

Julien Velcin* Walid Bechkit†

Année universitaire 2026–2027

1. Présentation

Ce projet permet à l'étudiant de réaliser un programme Python en passant par toutes les étapes de son cycle de vie : spécifications, analyse et conception, codage, vérification et maintenance. Il doit montrer qu'il est capable de mener à bien la réalisation d'une application ambitieuse, bien structurée et maîtrisée, qui pourra ensuite être utilisée, voire modifiée, par d'autres.

Le sujet est divisé en **trois parties**, la dernière offrant davantage de liberté à l'étudiant. Le rendu est demandé en **3 versions**, correspondant chacune à une évolution du projet (voir ci-dessous).

2. Date limite et rendu

Le rapport et le programme Python doivent être rendus le **dimanche 8 novembre 2026 à 23h59**. Le rapport (dont le contenu est détaillé plus bas) doit **impérativement** mentionner l'adresse du dépôt d'où peut être téléchargé le code de l'application. Il doit être déposé sur la page Moodle de l'enseignement **par chaque étudiant**.

3. Éléments à fournir

1. Un **rapport** (format `.pdf` uniquement, de taille inférieure à 5 Mo, d'une longueur indicative de **6 à 10 pages hors annexes**. Il rappelle la problématique du projet et en donne les spécifications, puis détaille l'analyse effectuée par les étudiants et la façon dont elle est implémentée (conception). Un soin particulier doit être apporté à l'explication de la **répartition des tâches** entre les étudiants : lors de l'évaluation, l'examinateur se réserve le droit d'interroger *individuellement* chaque étudiant et de prendre en compte ses réponses pour attribuer une note individuelle. Ce rapport **ne doit pas** contenir les lignes de code du logiciel. Il doit enfin présenter les **tests** effectués, une **conclusion** et des **perspectives**. Le détail est donné à la section 4.

2. Le lien du **dépôt GitHub** contenant le code source, clairement mentionné dans le rapport. Il comprendra l'ensemble des fichiers nécessaires à l'exécution du programme. Le code doit être correctement présenté (indentation), clair (noms lisibles) et commenté intelligemment. Les librairies éventuelles doivent être précisées afin de pouvoir exécuter facilement le programme. Le bon fonctionnement sera jugé sur une version **3.13** du langage Python. Trois versions du code doivent être fournies, si possible via le système de *tags* de GitHub :

   - **v1** : version correspondant à la première partie (socle de base) ;
   - **v2** : version correspondant aux deux premières parties (avec le moteur de recherche) ;
   - **v3** : version correspondant à toutes les parties (avec l'interface et l'extension).

*Auteur du sujet original.

†Responsable du cours

4. Le rapport

Le rapport doit comporter :

1. **Les spécifications** : normalement rédigées conjointement avec le client, elles décrivent en langage naturel ce que doit faire le programme. Ici, elles seront rédigées à partir des énoncés et des questions posées à l'enseignant.
2. **L'analyse** : comment les spécifications vont être réalisées. Elle comporte :
   - l'environnement de travail retenu (vous pouvez justifier votre choix) ;
   - les données identifiées dans les spécifications ;
   - un **diagramme de classes** (simplifié), c'est-à-dire :
     - les classes utilisées (nom et méthodes principales),
     - les relations d'*héritage* entre classes,
     - les relations d'*utilisation* entre classes.
3. **La conception** : comment l'analyse est implémentée, c'est-à-dire :
   - la manière dont les tâches ont été partagées entre les étudiants du projet ;
   - les algorithmes spécifiques à l'application et d'une certaine complexité (si nécessaire) ;
   - les problèmes rencontrés et comment vous les avez résolus ou contournés ;
   - un exemple commenté d'utilisation du programme, du début à la fin.
4. **La validation** : les procédures de test du logiciel, c'est-à-dire :
   - *tests unitaires* : comment les méthodes ont été testées individuellement (donnez quelques résultats) ;
   - *tests globaux* : via votre interface, quels cas particuliers avez-vous testés ?
5. **La maintenance** : quelques évolutions possibles du logiciel. Quelles fonctionnalités pourrait-on ajouter, et comment ? Pourquoi cela serait-il facile ou difficile ?

5. Notation du projet

La notation tiendra compte des éléments suivants :

1. **Fonctionnement général du programme** :
   - état d'avancement : jusqu'où les spécifications ont été réalisées sans bugs (un bug étant tout plantage inopiné du programme) ;
   - améliorations : les « plus » apportés aux spécifications initiales (pris en compte seulement si les spécifications de base sont d'abord réalisées) ;
   - les portions de code non valides (programme qui ne marche pas ou mal) sont mieux considérées si elles sont identifiées et commentées comme telles.
2. **Code** :
   - présentation générale (indentation, séparation des classes) ;
   - commentaires : en les lisant, on doit pouvoir comprendre le programme ;
   - qualité / efficacité du code au regard de la tâche réalisée (importance des tests).

3. **Dossier** :

   - analyse : qualité du choix des classes, prise en compte des mécanismes objets, qualité des algorithmes (mis en avant ici) ;
   - conception : correspondance avec la réalité, qualité de l'interface ;
   - présence de chaque partie, avec mise en avant des informations pertinentes ;
   - parties supplémentaires (tests et maintenance).

6. Remarques complémentaires

- Respectez le caractère **multi-plateforme** de Python : n'utilisez pas d'éléments spécifiques à un OS (par ex. pas de chemin absolu comme `C:\...\...`).
- Fournissez le nom des librairies nécessaires à l'exécution (vous pouvez générer un fichier de *requirements* : https://pip.pypa.io/en/latest/user_guide/#requirements-files).
- Il est *nettement* préférable que les données (en volume raisonnable) figurent aussi dans le dépôt (export de la table de données), pour ne pas avoir à re-télécharger les données via les API.
- Proposez a minima une petite **interface textuelle** pour interagir avec la base d'articles ; cela peut se faire avec un notebook ou une librairie plus ambitieuse (comme `dash`, parmi d'autres). **Attention :** il est interdit de rendre *tout* le code sous forme de notebook ! Cela doit être réservé à l'interface.
- Un programme qui fonctionne mais fait peu de choses sera mieux noté qu'un programme qui ne fonctionne pas du tout. Autrement dit, l'étendue des fonctionnalités *réellement opérationnelles* de l'interface est un critère important.
- L'utilisation de code d'autres auteurs (et libre de droit) est autorisée à condition : (a) d'en signaler **très clairement la source** — sous peine d'être accusé de plagiat, ce qui entraîne automatiquement une note de 0/20 a minima ; (b) que cela ne représente qu'une *partie* du code original que vous produisez.

  Cette remarque vaut aussi pour l'usage de Claude, ChatGPT et d'autres assistants du même type: ils sont autorisés dès lors que vous **signalez comment et où** vous les avez utilisés et que vous **comprenez parfaitement** le code produit. Ce point sera évalué lors de la démonstration ; vous risquez des pénalités importantes si vous ne maîtrisez pas votre code.

Bonne chance !